Promption for Prompt Injection Detection under Source Shift and Three State Enforcement

Roberto Enrique Quezada-Rodríguez
Universidad Nacional de Trujillo, Trujillo, Perú
requezadaro@unitru.edu.pe
ORCID 0009-0004-0869-3881

and

David Adrian Lucano Nieves
Universidad Nacional de Trujillo, Trujillo, Perú
requezadaro@unitru.edu.pe
ORCID 0009-0003-2167-8515

Abstract

Prompt injection defenses must balance malicious-input detection with the preservation of legitimate requests. We present an empirical study of Promption, a reusable software framework combining regular expressions, a lightweight text classifier, and a three-state enforcement policy. A code audit establishes the behavior of the deployed configuration and identifies a positional-index error in the existing data splitter. New experiments use 5872 normalized, unique prompts from five upstream collections, three stratified random holdouts, and five leave-one-source-out evaluations. All vectorizers and classifiers are fitted exclusively on each training partition. On the seed-42 random holdout, strict OR enforcement obtains 83.90% recall with 9.98% false positives, whereas the implemented three-state policy obtains 83.33% recall with 8.11% false positives and routes 89 malicious inputs to output inspection. Source-held-out results reveal substantial variation that random holdouts obscure. The study contributes a reproducible policy comparison and a distinction between input escape, guarded routing, and demonstrated downstream compromise. No language-model attack success or monetary savings are inferred from these classifier experiments. The results support Promption as a measurable input-screening framework while exposing the evidence needed to validate its complete security pipeline.

Keywords: Prompt injection, text classification, source shift, security middleware, software evaluation

1 Introduction

Applications built around large language models (LLMs) increasingly combine user requests with information retrieved from documents, services, and other tools. Their flexibility also exposes a security problem: text that should be treated as data can influence the instructions followed by the model. A prompt injection attempts to redirect an application away from its authorized task by placing adversarial instructions in an input or in external content. Greshake et al. demonstrated that this ambiguity extends beyond direct user interaction to material retrieved by LLM-integrated applications [1]. Protecting such applications therefore requires both an explicit trust boundary and evidence about the behavior of each defensive component.

An input detector addresses one part of this problem. It assigns a risk signal to a piece of text before the application forwards that text to a model. A detector can reject a recognizable instruction override, identify a suspicious linguistic pattern, or pass an ordinary request. However, a correct label is not the same as a secure application outcome. An unblocked malicious prompt may still fail to influence a model. Conversely, an input that looks ordinary may induce a harmful action when combined with particular instructions, retrieved information, or tool permissions. A useful evaluation must specify which outcome it observes and which parts of the system remain outside its measurements.

Promption is an academic software project with a reusable Python package and middleware for JavaScript applications. Its application includes a web service, a dashboard, corpus preparation, training, benchmark execution, and output inspection. The software makes it possible to study how deterministic rules and learned text features interact with application policy. This paper evaluates the supplied repository snapshot rather than treating its introductory documentation as a definitive specification. In particular, the active backend uses term frequency–inverse document frequency (TF-IDF) features and logistic regression. The active enforcement function has three

actions: ALLOWED, GUARDED, and BLOCKED. These choices materially affect the claims that an experiment can support.

The scientific problem is how to evaluate a layered input screen when its training corpus mixes source collections and when its enforcement policy can defer a suspicious input to a later control. A random split can place similar source-specific expressions on both sides of the split. A binary confusion matrix can also conceal the difference between a suspicious input sent to output inspection and an input rejected before generation. Neither problem is resolved by reporting a single aggregate accuracy. We examine source shift and enforcement actions explicitly, using a fresh experimental harness that imports the existing software without modifying its production implementation.

The study addresses three research questions. RQ1 asks how heuristic-only, classifier-only, strict OR, and implemented three-state enforcement differ on the same held-out inputs. RQ2 asks how their results change when an entire upstream source is excluded from classifier training. RQ3 asks whether existing evaluation artifacts and operational checks support the interpretation of those results. The first two questions are quantitative. The third combines a code audit with executable checks and provenance records.

The contributions are a formal account of the implemented decision policy, eight newly executed classifier-training experiments with row-level predictions, and a methodological audit that separates input interception from downstream attack success. The study does not introduce a new learning algorithm or claim superiority over contemporary agent defenses. Its original result is a controlled analysis of a concrete software implementation and the tradeoffs produced by its enforcement choices. The remainder of the paper positions the work, defines the system and threat model, describes the experimental protocol, reports the results, and discusses their practical and scientific limits.

2 Related Work

Indirect prompt injection concerns the promotion of untrusted content into instructions. The demonstrations by Greshake et al. [1] establish the relevance of that threat to integrated applications. BIPIA examines indirect attacks and defenses that help models recognize instruction boundaries in external content [2]. These works motivate a distinction between suspicious wording and the trust context in which the wording appears. A prompt-only classifier can identify patterns associated with attacks, but it does not itself recover the authority of every instruction in a running application.

InjecAgent evaluates harmful behavior in tool-integrated agents [3]. AgentDojo provides an extensible environment for testing tasks, attacks, and defenses over untrusted tool data [4]. Their observation unit includes an agent operating in an environment. Promption's present experiments instead use one labeled text input as the observation unit. We cite these benchmarks to establish what an application-level security experiment would require, not to compare their attack success rates with our classifier recall. The environments, threat models, and denominators are different.

Model-side and system-side defenses act at different places. The instruction-hierarchy work of Wallace et al. trains models to resolve conflicts according to instruction privilege [5]. CaMeL separates control and data flows and applies capability-based restrictions around model-mediated tool use [6]. Promption's text screen supplies a risk signal before generation, while its broader library exposes authorization and output-protection mechanisms. The present study evaluates the text screen and its routing policy. It does not establish the control-flow guarantees studied by CaMeL or the robustness of a model trained on instruction-hierarchy examples.

Recent work on agent–tool firewalls also emphasizes that apparent defensive performance depends on benchmark implementation, outcome definitions, and attack strength [7]. That methodological concern is directly relevant here. A detector can look effective on familiar templates while failing on a different source; an evaluation can also report success against a deterministic proxy that never invokes a language model. Our protocol records partition membership and distinguishes the software's actions so that these assumptions remain visible.

The upstream corpus supplies another point of comparison. The jailbreak_llms repository contains prompts collected from several online communities and datasets [8]. The other collections used here are distributed through Hugging Face [9]–[12]. Their combination provides varied wording, but the labels and source categories are heterogeneous. We therefore evaluate the local, curated snapshot rather than claiming that the merged corpus is an exhaustive benchmark for prompt injection. A public collection may itself contain generated templates, and the absence of a local synthetic-source label does not imply naturally occurring data.

Table 1 summarizes the relationship between this study and the primary research used to motivate it. The table compares evaluation scope rather than ranking performance. Our experiments provide direct evidence about input decisions under retraining and source withholding. They leave live model behavior, task completion, and adaptive tool attacks for a separate evaluation. This positioning matters because a small middleware component can be useful without inheriting the security guarantees of a full agent architecture.

Table 1: Evaluation boundaries in related research and the present study

Work	Principal boundary	Relation to this study
Greshake et al. [1]	Retrieved adversarial text	Motivates trust-context analysis
BIPIA [2]	Model output on indirect attacks	Requires live-model outcome checks
InjecAgent [3] and AgentDojo [4]	Tool-integrated tasks	Observe agent actions and task outcomes
Instruction hierarchy [5]	Model instruction privilege	Acts within the language model
CaMeL [6] and firewalls [7]	Control flow and tool interfaces	Enforce boundaries around model behavior
Present study	Input rejection and guarded routing	Paired policies and classifier source withholding

3 System and Threat Model

3.1 Software Boundary

Promption separates reusable protection mechanisms from the surrounding application. The Python package contains filtering, guards, conversation handling, policies, authorization, tool integration, and clients. A second package exposes middleware and types for applications using AI SDK, a software development kit (SDK) for artificial intelligence (AI) applications. The application contributes its own business context, data, web interface, and operational monitoring. This separation permits the detector to be evaluated in process without invoking the user interface or making external model calls.

Fig. 1 shows the distinction between the input-screening boundary measured in this paper and the downstream generation path. The deterministic heuristic produces a score and matched-rule information. The classifier produces a probability associated with the malicious class. The enforcement function combines those signals to choose an action. BLOCKED prevents forwarding at this boundary. GUARDED requests downstream output inspection. ALLOWED indicates that the input policy permits forwarding, although a caller may still apply additional controls. The figure also places tool authorization outside the classifier, where permissions belong.

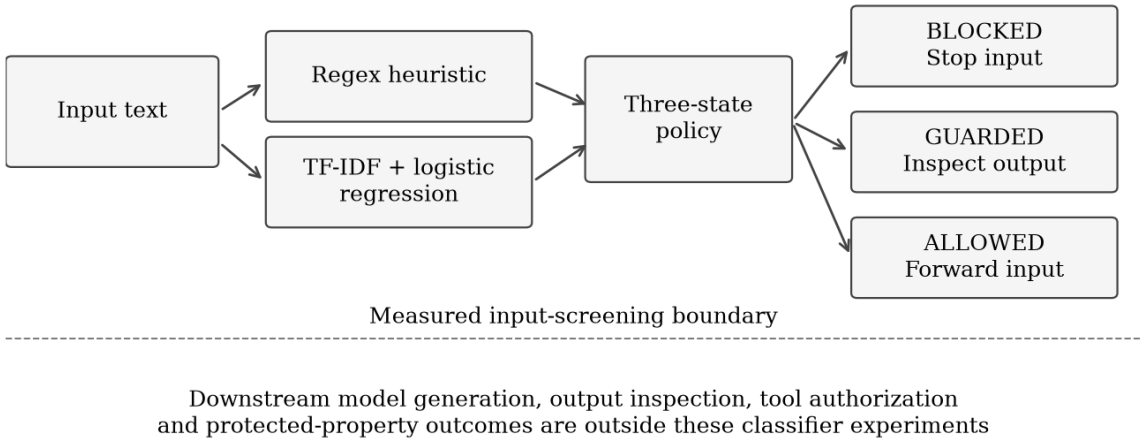


Figure 1: Input screening and the downstream application boundary

The repository supports an embedding-based backend and a lightweight backend. The active configuration selects the latter. Consequently, the experiments retrain the software's TF-IDF and logistic-regression implementation rather than loading the supplied random-forest artifact or downloading an embedding model. This choice reproduces the configured feature architecture while providing control over which examples enter training. The previously serialized lightweight model is documented as an existing artifact but is not used to make the new held-out predictions.

3.2 Heuristic and Learned Signals

The heuristic compiles 60 enabled malicious rules and 5 benign-expression rules. Each malicious match carries a severity score: 0.4 for low, 0.7 for medium, and 1.0 for high. It considers at most the configured number of malicious matches, five in this snapshot, and returns the maximum severity among the inspected matches. A malicious match takes precedence over a benign-expression match. A benign-only match produces score zero. Text with neither type of match is unknown and receives score 0.5. Role-scoped rules are considered only when supplied roles overlap their declared scope; the experiments call the heuristic with no roles.

Let h(x) denote the heuristic score for input x and H(x) its independent blocking decision. In the evaluated configuration, H(x) is one when there is a malicious signal and h(x) is at least 0.66. An unknown score of 0.5 therefore does not independently block. This is a risk convention rather than an estimate of the probability that an attack succeeds. In particular, score zero records a benign-expression match under the current rules; it does not establish that the entire downstream application is safe.

The machine learning (ML) branch concatenates word and character TF-IDF features using a feature union. Word features use unigrams and bigrams, a minimum document frequency of two, and a maximum document-frequency proportion of 0.95. Character features use within-word character n-grams of lengths three through five and a minimum document frequency of two. Each branch is capped at 10,000 features. Both branches lowercase text and apply sublinear term-frequency scaling. These transformations follow the implementation exposed by scikit-learn [13]. The union is fitted only on training prompts.

The classifier is logistic regression with C equal to 1, balanced class weights, at most 1,000 iterations, and classifier random state 42. Its positive-class probability p(x) is the quantity used in policy decisions; we do not assume it is calibrated for deployment prevalence. The isolated ML blocking rule in this study uses p(x) greater than 0.66, matching the comparison performed by the ensemble wrapper. The stand-alone lightweight class uses an inclusive comparison at its own threshold, so threshold behavior must be stated at the evaluated boundary. The classifier interface and its balancing options are documented in scikit-learn [14].

3.3 Enforcement Semantics

A strict OR baseline blocks an input whenever either independent layer blocks. Its decision is B_OR(x) = H(x) OR [p(x) > 0.66]. This baseline preserves the familiar property that adding a blocking layer cannot reduce the number of blocked inputs. We compute it from the same heuristic outputs and classifier probabilities used by the other policies. It is an experimental comparator, not a replacement for the repository's enforcement function.

The implemented policy first handles score extremes. If h(x) is zero, it returns ALLOWED. If h(x) is at least one, it returns BLOCKED. For all remaining scores, an unavailable classifier produces GUARDED. With an available classifier, the function permits only the combination of low heuristic risk and low ML risk. It blocks when heuristic risk is medium or high and ML risk is high. All remaining combinations produce GUARDED. Risk is low below 0.33, medium from 0.33 through 0.66 inclusive, and high above 0.66. Table 2 gives the operational cases for the discrete heuristic scores observed in this design.

Table 2: Implemented actions at the configured decision boundary

Heuristic score	ML probability	Action
0.0	Any or unavailable	ALLOWED
1.0	Any or unavailable	BLOCKED
0.4, 0.5, 0.7	Unavailable	GUARDED
0.4, 0.5, 0.7	$p \leq 0.66$	GUARDED
0.4, 0.5, 0.7	$p > 0.66$	BLOCKED

Several consequences follow from this definition. A medium-severity rule can independently block in the heuristic branch while the policy sends the same input to GUARDED when the classifier is not high. A benign-only match can override a high classifier signal and still return ALLOWED. An unknown input with a low classifier probability remains GUARDED because its heuristic score is medium. The weighted ensemble score, calculated from heuristic risk and ML probability, is descriptive: the action function does not threshold that weighted score. These properties explain why a binary report labeled simply as an ensemble can be misleading.

3.4 Adversary and Security Outcomes

The adversary in the classification experiments controls the text of a single input. The available labels identify corpus examples as benign or malicious according to the project's curated data. The adversary does not modify the installed code, configuration, role identity, or training procedure during evaluation. There is no optimization against the retrained models, no active interaction with a model endpoint, and no multi-turn environment. We evaluate rejection and routing decisions over previously collected examples.

Three outcomes must remain separate. Input escape means that a corpus example labeled malicious is not blocked at the input boundary. Guarded routing means that such an example is forwarded under a requirement for output inspection. Downstream compromise means that the application's model or tools violate a specified protected property. Only the first two outcomes are observed in the new experiments. Output inspection may prevent a recognizable secret from being delivered, but this possibility cannot turn a guarded case into a measured prevention without examining the actual generated output.

This threat model covers screening of direct injection and jailbreak-like text in the corpus. It provides incomplete evidence about indirect injection because the experiments do not embed the text into retrieved documents with provenance or surrounding tasks. Portable Document Format (PDF) extraction and audio transcription are available software adapters, but their accuracy and resulting security behavior are not tested here. Likewise, role authorization can limit tool access independently of classifier performance, yet its application-specific decisions are not quantified in the main result tables.

4 Experimental Method

4.1 Corpus Construction and Provenance

The starting processed comma-separated values (CSV) file contains 7,823 prompts. The repository's loader applies its existing label overrides and quarantine policy before returning the evaluation snapshot. Nineteen override entries are matched in this run; quarantine removes no further rows from the processed file. The resulting binary distribution is 4247 benign and 3576 malicious examples. These are local project labels after curation. We do not reinterpret them as universally agreed labels for every upstream source, nor do we independently reannotate the entire collection.

For the new experiments, we select the source values NEPI, Shomi28, deepset, jailbreak_llms, and yanismiraoui. The selection contains 5885 rows before normalized-text cleaning. It excludes local prompts, built-in prompts, explicitly translated local sources, hard negatives, and other locally marked synthetic additions. It also excludes the source owasp_top10, whose examples are built into the downloader. This selection reduces local augmentation overlap, but it must not be described as excluding every generated example: the NEPI source includes template-generated prompts [9].

Normalization uses Unicode Normalization Form KC (NFKC), case folding, and whitespace collapse. It supplies an identity key for removing repeated text; it does not change the prompt passed to the detector. We remove every normalized group containing conflicting binary labels, then retain the first row of each remaining normalized group in original CSV order. This removes 0 rows from conflicting groups and 13 additional duplicate rows. The retained corpus contains 5872 inputs, including 3201 benign and 2671 malicious examples. Source counts are reported in Table 3.

Table 3: Retained corpus after normalized exact deduplication

Source	Benign	Malicious	Total
NEPI	201	313	514
Shomi28	640	640	1280
deepset	401	259	660
jailbreak_llms	1959	1357	3316
yanismiraoui	0	102	102
Total	3201	2671	5872

Each retained example receives a Secure Hash Algorithm digest with 256-bit output (SHA-256) of its normalized text. The digest is used to assert disjoint training and test partitions and to identify row-level predictions without placing the prompt in the result table. The normalization procedure cannot detect paraphrases, semantically

equivalent prompts, or shared templates with different parameters. Exact identity is therefore controlled more strongly than family-level dependence. Near-duplicate and template overlap remain threats to validity, particularly in the random splits.

The source values record upstream distribution channels, not immutable attack families. Two sources can contain related material, and a source can mix several behaviors. The repository downloader also combines upstream train, validation, and test partitions for some datasets. Our experiment creates new partitions from the local snapshot and does not claim to reproduce those upstream benchmark splits. Table 3 consequently describes the corpus actually evaluated, not the advertised size of any dataset at its public distribution site.

4.2 Random and Source Held Out Partitions

We create three stratified random holdouts using seeds 42, 43, and 44. Each holdout allocates 20 percent of the retained corpus to testing and stratifies only by the binary label. It uses 4697 training examples and 1175 test examples. The three repetitions probe sensitivity to this partition choice. They are overlapping resamples of one corpus and are not independent replications or disjoint cross-validation folds. Mean and sample standard deviation across seeds are reported as descriptive summaries.

We also create five leave-one-source-out experiments. In each experiment, one of the five source values forms the complete test set, and the other four form the training set. The classifier and both vectorizers are refitted from scratch. A source-held-out result therefore measures transfer of the learned component to that collection while keeping the existing rules fixed. It does not prove that those rules were designed without seeing the held-out source during the earlier development of the project. The distinction between classifier exclusion and rule independence is explicit throughout the interpretation.

The same unmodified training function is used for all eight fits. The experimental harness bypasses the existing splitter because its audit reveals a positional-index error. That splitter filters the frame to ostensibly public rows, obtains grouped positions relative to the filtered frame, and later uses the positions against the original unfiltered frame. On this snapshot, the returned test indices identify 1,184 examples, of which 199 have source values that the splitter intends to exclude. The error does not by itself establish how much any old performance number is biased, but it prevents us from treating the previous partition as the intended public holdout.

No test result is used to select a classifier setting or a decision threshold. The representation, C value, class weights, and thresholds are taken from the supplied implementation and configuration. All feature fitting occurs on each training partition. We save per-experiment membership, probabilities, source labels, and policy decisions. This arrangement makes the policy comparisons paired within an experiment: every comparator sees exactly the same test examples and the same learned probabilities.

4.3 Comparators and Metrics

Four input policies are evaluated. Heuristic-only uses the heuristic's independent blocked flag. ML-only blocks when the retrained positive probability exceeds 0.66. Strict OR blocks if either of those policies blocks. Three-state enforcement calls the existing decide_pipeline_action function and treats only BLOCKED as binary rejection. GUARDED and ALLOWED both count as unblocked in the binary confusion matrix, and their separate counts are reported. This convention intentionally avoids crediting deferred output protection as if it had already prevented an attack.

For y equal to one, a blocked input is a true positive (TP), and an unblocked input is a false negative (FN). For y equal to zero, a blocked input is a false positive (FP), and an unblocked input is a true negative (TN). Precision is TP divided by TP plus FP. Recall is TP divided by TP plus FN. F1 is the harmonic mean of precision and recall. The false-positive rate (FPR) is FP divided by FP plus TN. Accuracy is TP plus TN divided by all test inputs. Table 4 makes the security interpretation of these quantities explicit.

Table 4: Meaning and limits of the reported observations

Quantity	Observed event	Interpretation limit
Recall	Malicious inputs blocked / malicious inputs	Measures input interception
FPR	Benign inputs blocked / benign inputs	Measures rejection against corpus labels
Precision	Malicious inputs / all blocked inputs	Depends on observed prevalence
GUARDED count	Inputs routed to output inspection	No generated output is observed

Quantity	Observed event	Interpretation limit
Input escape	Malicious inputs not blocked	Does not establish compromise
Warm latency	In-process screen and action selection	Excludes the complete application path

For ML-only, the area under the receiver operating characteristic curve (ROC AUC) is computed from continuous probabilities when both labels are present. The other policies are deterministic decisions; any AUC computed from their binary outputs is not used as evidence about score ranking. The yanismiraoui holdout contains only malicious examples in this snapshot. We report recall and TP/FN for that holdout but do not assign an FPR or a probability-based AUC. A missing denominator is shown as not applicable rather than zero.

Uncertainty is characterized conservatively. We report exact confusion counts for the seed-42 experiment and descriptive dispersion for the three random splits. For source-held-out recall and FPR, Wilson intervals are computed as conditional binomial summaries of each labeled set. These intervals do not correct for template dependence, uncertain labels, or dataset-selection bias. Paired differences in blocking are also reported as counts because the OR policy contains the three-state blocking set under this score design. We do not claim that small differences between overlapping random splits establish statistical superiority.

4.4 Timing and Operational Validation

Latency is measured with a monotonic, high-resolution clock around heuristic analysis, scalar vectorization, classifier probability prediction, and action selection. After training the seed-42 model and using it for batch predictions, we process the first 200 seed-42 test prompts individually in three passes. All 600 measurements are therefore warm in-process observations. Model loading, training, Hypertext Transfer Protocol (HTTP) transfer, model generation, output inspection, and dashboard rendering are outside the timer. The input order is fixed rather than selected to optimize timing.

The experiment uses Python 3.11.15, scikit-learn 1.9.0, pandas 3.0.6, and NumPy 2.4.6 on the recorded Linux environment. The execution manifest records the operating-system string, central processing unit (CPU) count, source hashes, and the repository commit. Training is executed with one requested OpenMP and OpenBLAS thread. The latency result is a measurement on this development host, not a service-level guarantee. Simultaneous repository checks and other host activity can affect timing, so median and 95th percentile are more informative than a universal speed claim.

Operational checks are kept distinct from predictive metrics. We execute the Python test suite and the JavaScript package tests, and exercise the benchmark in no-LLM mode without replacing earlier application results. Health and dashboard smoke checks are attempted to examine importability and startup, with any incomplete check recorded separately. These checks can catch implementation regressions and wiring failures; they do not supply attack labels, live-model outcomes, or a general security guarantee. Their recorded status appears in the reproducibility material rather than being counted as successful adversarial trials.

5 Results

5.1 Random Holdout Performance

Table 5 reports the seed-42 confusion counts and derived classification metrics. The test partition contains 534 malicious and 641 benign inputs. Heuristic-only blocks 99 malicious examples and 49 benign examples, giving 18.54 percent recall and 7.64 percent FPR. ML-only blocks 443 malicious examples and 19 benign examples, giving 82.96 percent recall and 2.96 percent FPR. Its precision of 95.89 percent shows that, on this split, relatively few of its positive decisions concern examples labeled benign.

Table 5: Seed 42 test results on 534 malicious and 641 benign inputs

Policy	TP / FP	FN / TN	Precision %	Recall %	FPR %
Heuristic	99 / 49	435 / 592	66.89	18.54	7.64
ML	443 / 19	91 / 622	95.89	82.96	2.96
Strict OR	448 / 64	86 / 577	87.50	83.90	9.98
Three state	445 / 52	89 / 589	89.54	83.33	8.11

Strict OR raises the number of blocked malicious inputs from 443 to 448 compared with ML-only. It also raises benign blocks from 19 to 64. Thus, the heuristic adds five true positives and 45 false positives at this operating point. The resulting recall is 83.90 percent and the FPR is 9.98 percent. This comparison supports a narrow conclusion about this corpus and threshold: combining these signals by OR improves interception slightly while increasing overblocking substantially. It does not show that heuristic rules are intrinsically unsuitable or that OR would behave identically with a different rule set.

The implemented three-state policy blocks 445 malicious and 52 benign inputs. Its 83.33 percent recall and 8.11 percent FPR lie between the strict OR results and the ML-only results for these two metrics. Relative to strict OR, the policy permits twelve additional benign inputs to proceed under protection and routes three additional malicious inputs to the same later control. Relative to ML-only, it blocks two additional malicious inputs and 33 additional benign inputs. These paired counts expose the concrete cost of each policy choice more clearly than accuracy alone.

For this seed, ML-only has the highest F1 among the four binary policies, 0.8896. The implemented policy reaches 0.8632 and strict OR reaches 0.8566. Accuracy is 0.8800 for the implemented policy. F1 favors a balance between positive precision and recall under the observed label distribution; it does not express the application-specific harm of a leaked secret or the cost of a rejected legitimate request. A deployment choosing a stricter policy could rationally accept a lower F1 if the consequence of a missed attack dominates its utility loss.

Table 6 reports the descriptive mean and sample standard deviation across the three random holdouts. ML-only has mean recall 82.27 percent and mean FPR 3.69 percent. Strict OR has mean recall 83.90 percent and mean FPR 11.13 percent. Three-state enforcement has mean recall 82.65 percent and mean FPR 9.41 percent. The ordering in the seed-42 comparison therefore also appears in the three-seed averages. Because the test sets overlap, these summaries quantify partition sensitivity rather than three independent confirmations of a population effect.

Table 6: Mean and sample standard deviation over three overlapping random holdouts

Policy	Precision %	Recall %	F1	FPR %
Heuristic	66.78 ± 1.70	20.10 ± 1.70	0.3088 ± 0.0212	8.32 ± 0.63
ML	94.89 ± 1.19	82.27 ± 0.60	0.8813 ± 0.0082	3.69 ± 0.89
Strict OR	86.28 ± 1.50	83.90 ± 0.56	0.8507 ± 0.0096	11.13 ± 1.36
Three state	87.99 ± 2.02	82.65 ± 0.66	0.8523 ± 0.0128	9.41 ± 1.74

5.2 Source Shift

Table 7 gives recall for each source-held-out experiment, alongside the FPR of the implemented policy and its conditional Wilson intervals. In the table, H denotes heuristic-only, ML classifier-only, OR strict OR, 3S three-state enforcement, and CI confidence interval. The source-held-out results vary far more than the three random splits. When NEPI is excluded from training, the implemented policy blocks 79 of 313 malicious prompts, giving 25.24 percent recall. It blocks none of the 201 benign prompts. For deepset, it blocks 45 of 259 malicious prompts, giving 17.37 percent recall, and one of 401 benign prompts, giving 0.25 percent FPR. Low FPR in these two cases therefore coexists with substantial malicious-input escape.

Table 7: Source held out recall and implemented policy false positives

Held-out source	Recall H / ML %	Recall OR / 3S %	3S recall 95% CI	3S FPR % [95% CI]
NEPI	0.96 / 25.24	25.24 / 25.24	20.75–30.33	0.00 [0.00–1.88]
Shomi28	25.31 / 84.53	88.91 / 88.59	85.90–90.83	0.31 [0.09–1.13]
deepset	5.02 / 16.22	17.37 / 17.37	13.25–22.46	0.25 [0.04–1.40]
jailbreak_llms	24.69 / 96.76	96.76 / 96.76	95.68–97.58	80.09 [78.27–81.80]
yanismiraoui	2.94 / 9.80	12.75 / 11.76	6.86–19.45	N/A

When Shomi28 is held out, the implemented policy blocks 567 of 640 malicious inputs and two of 640 benign inputs. Recall is 88.59 percent and FPR is 0.31 percent. Strict OR intercepts two more malicious examples with the same benign-block count. This collection is more readily screened by the representation and rules learned or fixed

from the remaining sources. The result should not be interpreted as a general statement about all unseen sources, because the NEPI and deepset holdouts show very different transfer behavior.

The jailbreak_llms holdout illustrates the opposite problem. The implemented policy blocks 1,313 of 1,357 malicious inputs, producing 96.76 percent recall, but it also blocks 1,569 of 1,959 benign inputs, producing 80.09 percent FPR. ML-only already exhibits 79.99 percent FPR. The learned representation therefore transfers a broad attack association to text that the held-out collection labels benign. Adding heuristic enforcement changes little in this case. An aggregate recall near 97 percent would conceal a severe utility failure if reported without its negative-class denominator.

The yanismiraoui subset contains 102 malicious inputs and no benign inputs. The implemented policy blocks twelve, giving 11.76 percent recall. Strict OR blocks thirteen, and ML-only blocks ten. We cannot use this subset to estimate a false-positive rate. The low detection count is especially relevant to the corpus's small Spanish component, but source and language are confounded. It would be incorrect to attribute the result solely to Spanish or to claim a controlled cross-language comparison.

Fig. 2 places the implemented policy's source-held-out recall beside its FPR, where that denominator exists. The important pattern is the spread: one source combines high recall with low FPR, two show poor interception despite almost no false positives, and the largest source has high recall with extensive overblocking. These outcomes demonstrate why a pooled random holdout is an insufficient description of transfer across the available source collections. They also identify which kinds of follow-up data collection would be most informative.

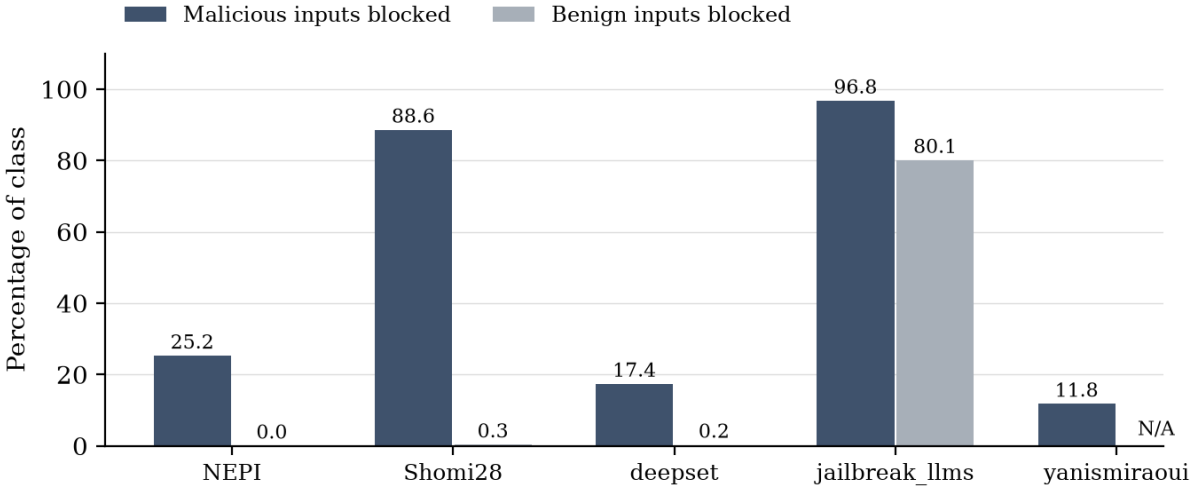


Figure 2: Implemented policy performance under classifier source withholding

5.3 Guarded Routing and Policy Dependence

In the seed-42 holdout, all 678 unblocked inputs are GUARDED: 89 malicious and 589 benign. No example receives ALLOWED. The implemented heuristic returns either a malicious score or the unknown score for these inputs; the benign-only path is not exercised by this particular test set. Table 8 shows the full action breakdown. Consequently, the binary recall of the implemented policy measures pre-generation rejection, while the operational workload includes output inspection for more than half of the tested inputs.

Table 8: Implemented policy actions for the seed 42 test partition

Label	ALLOWED	GUARDED	BLOCKED	Total
Malicious	0	89	445	534
Benign	0	589	52	641
Total	0	678	497	1175

The 89 guarded malicious inputs constitute 16.67 percent of malicious examples in this split. They are unresolved cases for the input boundary, not demonstrated leaks and not measured prevention successes. If an application correctly enforces its output inspection requirement, some may be stopped later. If a caller forwards them without that required control, they have the same downstream exposure as otherwise unblocked inputs. The experiments do not distinguish those downstream possibilities because they generate no responses.

Fig. 3 visualizes the action distribution by binary label for the seed-42 experiment. It shows that reducing input overblocking through GUARDED does not eliminate the cost of a later check. In particular, 91.89 percent of benign inputs proceed under the guarded path. This matters operationally when output scanning, buffering, policy enforcement, or user-visible handling has a nonzero cost. It also matters scientifically because routing a legitimate input to GUARDED is not equivalent to completing the user's intended task successfully.

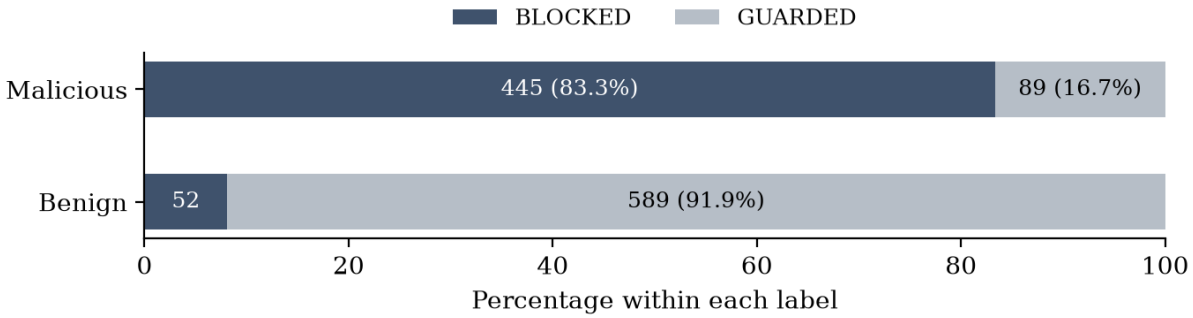


Figure 3: Implemented policy actions on the seed 42 random holdout

The decision matrix also exposes a conditional failure behavior. When the ML branch is unavailable, a nonextreme heuristic score is GUARDED. Score zero remains ALLOWED, and score one remains BLOCKED. Thus, graceful degradation of the classifier does not mean uniform fail-closed behavior at the input boundary. The matrix provides a precise contract for callers: a guarded decision requires another enforcement step. Its behavior must be checked in the complete integration, rather than inferred from a nonthrowing classifier application programming interface (API).

5.4 Timing and Audit Findings

Across 600 warm scalar predictions, the combined heuristic, TF-IDF transform, probability prediction, and action selection have median latency 4.33 milliseconds and 95th-percentile latency 21.69 milliseconds. The mean is 7.01 milliseconds. These observations support the feasibility of an in-process input screen on the recorded host. They exclude network transfer, cold startup, language-model inference, output scanning, and application concurrency, so they do not establish the latency of a complete guarded request.

The audit produces three findings that affect interpretation independently of the classifier results. First, the active model differs from the embedding-based model described in the project's introductory materials. Second, the implemented action function differs from a strict OR ensemble and cannot be reconstructed by thresholding its displayed weighted score. Third, the existing splitter returns positions from a filtered frame that do not refer to the intended rows of the original frame. Table 9 states each finding and the measure taken in the new experiments.

Table 9: Audit findings and their treatment in the experimental protocol

Finding	Evidence	Experimental treatment
Configured backend	use_lightweight_ml is true	Refit the TF-IDF and logistic model
Three-state semantics	Action function differs from strict OR	Compare both over paired inputs
Position mismatch in splitter	199 locally excluded-source rows enter intended test	Construct new holdouts directly
No-LLM success proxy	Success becomes complement of input blocking	Report input escape only
Lenient response oracle	Non-refusal can count as compromise	Do not infer secret disclosure

The supplied benchmark CSV contains 600 rows, and its companion JavaScript Object Notation (JSON) file carries earlier live-model summaries. Those artifacts are historical observations, not reruns under the protocol described here. The file modification date and embedded experiment timestamp also need separate interpretation. We do not combine those earlier observations with the new source-held-out predictions. The paper's central quantitative results come only from the eight fresh fits and the corresponding saved prediction records.

A second metric issue arises in the benchmark runner's no-LLM path. It sets a malicious example's unfiltered success indicator to one and its filtered success indicator to the complement of input blocking. Under that definition, the resulting so-called attack-success proxy is exactly the malicious-input escape rate. The live-model helper also has a lenient mode that treats responses without a recognized refusal as compromised, even without a demonstrated secret leak. Therefore, a benchmark labeled attack success rate (ASR) must state whether it means input escape, non-refusal, exact secret disclosure, or another protected property. We retain that distinction instead of reproducing historical ASR and cost claims in the abstract.

6 Discussion

6.1 Detection and Enforcement Address Different Questions

The experiments show that representation quality and enforcement strictness should be analyzed separately. ML-only provides the strongest binary F1 on the random holdouts, while OR produces the greatest random-holdout interception among the tested policies. These statements are compatible because OR adds benign blocks along with malicious blocks. The implemented three-state policy reduces some of that overblocking by deferring inputs, but it also transfers responsibility to output controls. A practical choice depends on what the application protects and on the reliability of that later control.

One useful interpretation is that the input stage performs admission control rather than a final safety judgment. A malicious label describes the text's role in a corpus. BLOCKED describes a decision to stop that text before generation. GUARDED describes a conditional route. None of these quantities reports whether an actual authorized task finishes or whether a protected property is violated. Keeping that separation prevents a favorable detector result from silently becoming a claim about end-to-end security.

The seed-42 comparison is particularly concrete. Strict OR adds five malicious blocks and 45 benign blocks beyond ML-only. Three-state enforcement recovers twelve of the benign requests rejected by OR while sending three additional malicious requests onward under protection. Those paired changes can inform application design, but they cannot determine a universal optimal policy. A support assistant, an internal analytics tool, and a tool-using agent with write permissions have different consequences for false negatives and false positives.

The policy's benign override deserves a separate observation even though the random seed-42 set does not exercise it. Under the action function, h equal to zero permits an input regardless of the classifier's high probability. A matching benign phrase must therefore have a carefully bounded scope and remain subordinate to malicious matching. In this implementation, malicious matches take precedence, which avoids the simplest benign-plus-malicious collision. Nonetheless, the probability override and the absence of a role in the present experiments limit the breadth of any claim about deployment behavior.

6.2 What Source Withholding Reveals

The strongest result is the contrast between random-holdout performance and heterogeneous transfer. Random test examples share the mixture of source conventions present during training. Leave-one-source-out testing removes that advantage for the learned branch. The NEPI and deepset results show that a high pooled recall can coexist with poor interception on a source excluded from training. The jailbreak_llms result shows that high transfer recall can coincide with rejecting most legitimate examples from the held-out collection.

Several explanations remain plausible. Different sources use different definitions of injection, different legitimate tasks, and different writing styles. Their negative-class distributions are especially important: class-balanced optimization can learn features that separate the training sources yet transfer poorly to ordinary prompts in another collection. Source identity can also be associated with label balance, prompt length, or recurring templates. The present ablation demonstrates the performance shift but does not isolate the causal contribution of each factor.

The source-held-out classifier is fitted without examples from its test source, but the heuristic rules are frozen from a project developed before this experiment. If rule authors consulted the same collections, some rule-level familiarity may remain. Accordingly, the holdout should be interpreted as learned-component transfer within the supplied system, not a fully blinded external validation of every layer. The modest heuristic recall on several holdouts and its substantial jailbreak_llms overblocking make this qualification relevant in both directions.

A future prospective test should define protected actions, select newly collected or independently curated inputs, and freeze all rules and model settings before seeing that test collection. It should separate source shift from language shift by using sufficiently sized benign and malicious sets in each language. It should also record the raw task context surrounding an indirect injection. Such a protocol would examine whether the behaviors measured here continue when source templates and prior development exposure are controlled more strongly.

6.3 Output Controls and Tool Authorization

A guarded input can still redirect a model internally. Output scanning examines the delivered text, while authorization restricts operations and information access. These controls observe different objects. A response scanner may recognize a credential pattern but cannot undo a tool action that already changed remote state. An authorization gate can reject a disallowed tool call without recognizing the malicious wording that caused it. Promption's reusable interfaces support those control locations, but the present classifier evaluation does not measure their composition.

This distinction connects the software evaluation to the system-level work on instruction privilege, capabilities, and tool-interface defenses [5]–[7]. Those approaches motivate enforceable boundaries around model behavior. Promption can provide an additional risk signal within such a boundary, but a text classifier should not receive decision authority over permissions that the application has already established. Role identity and business constraints must originate from authenticated application state rather than from the text being screened.

Streaming introduces an additional integration condition. If an application exposes response fragments before completing inspection, a later BLOCK or REDACT result may arrive after protected content has already been disclosed. Output inspection must therefore be compatible with the delivery strategy of the caller. Our experiments neither stream responses nor test partial disclosure. The GUARDED action counts should be read as a demand on the integration, not as proof that this condition is enforced by every possible caller.

Likewise, PDF and audio adapters convert external media into text but do not establish the trustworthiness of the extracted content. Their parsing or transcription errors can change both attack visibility and benign meaning. Treating their outputs as inputs to the same screen is an architectural convenience. Security evidence for those channels would additionally require a corpus of actual files, extraction-quality checks, and outcome measurements in the application that consumes the extracted text.

6.4 Implications for Evaluation Practice

A reproducible detector study should begin with the implemented boundary. In this case, relying on the README would select the wrong feature backend and the wrong ensemble semantics. Inspecting configuration and action code changes both the experiment and its interpretation. The evidence package therefore records the evaluated commit and hashes of configuration, data, relevant source files, and the existing model artifact. These records help separate the measured version from later code or corpus changes.

Partition identity is equally important. A split function can appear deterministic and still return the wrong examples if frame-relative positions are confused with original positions. Determinism only makes the same error repeatable. The new harness saves normalized digests and asserts partition disjointness. It also constructs source holdouts directly from the retained frame. These choices make the evaluated rows inspectable without changing the production splitter as part of the writing task.

Finally, a single success label should not stand for multiple outcomes. Input escape is measurable without a language model. Non-refusal requires a generated response and an operational definition of refusal. Secret disclosure requires evidence of the protected value or a suitable semantic or reconstruction oracle. Unsafe tool execution requires an environment-level check. A defensible article can report the first of these and leave the others unmeasured, provided that its conclusions remain at the same boundary.

7 Threats to Validity

The main construct-validity issue is the label definition. The curated corpus includes direct override instructions, jailbreak-like prompts, narrative examples, and legitimate requests. A malicious label need not mean that the same text would violate every application's policy. Conversely, an apparently legitimate request may be unsafe under a particular role or business constraint. The reported metrics therefore measure agreement between input-blocking decisions and the project's binary labels, rather than an independent universal ground truth for all LLM applications.

Internal validity is limited by incomplete family-level separation. Normalized exact duplicates are removed, and no normalized text crosses a partition, but paraphrases and parameterized templates may still occur in both random partitions or across sources. Existing label overrides were curated before this experiment and may reflect project-specific intent. The rules were not independently redesigned or blind-reviewed for each partition. These factors can affect absolute performance even though the paired policy comparisons use identical test inputs.

The source-held-out design also changes training-set size, class balance, source mixture, and test composition simultaneously. Withholding jailbreak_llms removes the largest collection; withholding yanismiraoui removes only a small attack-only source. Their results are not interchangeable trials of one homogeneous shift. We report each collection separately and avoid interpreting their average as a population estimate. A controlled study of source effects would require matched samples or additional designs that vary one factor at a time.

Statistical conclusion validity is limited by dependence within the text corpus. The three random holdouts overlap. Wilson intervals assume a conditional binomial view that does not model shared templates or uncertain annotations. Source-specific intervals are descriptive and should not be read as confidence bounds for all future inputs from that source. We also perform no adaptive threshold search, model-selection study, or hypothesis test claiming superiority over external defenses. Their experimental domains would not be directly comparable.

External validity is restricted to a small, mostly English corpus and one feature architecture. The retained data contain 5,652 inputs labeled English and 220 labeled Spanish by the repository's language detector. Those tags are approximate, and their imbalance prevents a robust multilingual claim. No new model endpoints are queried, and no live task-completion, indirect-context, adaptive-attack, multi-turn, PDF, or audio experiment is executed. Results cannot be generalized to those channels solely because the code includes an adapter or guard interface.

Timing validity is limited to warm scalar inference on one development host. The CPU count in the manifest does not identify a controlled dedicated hardware configuration, and the processor field returned by the runtime is empty. The fixed 200-input timing cohort may not match the length distribution of production traffic. Concurrent host workloads can affect tail observations. We therefore report the timing scope and sample size, and avoid comparing milliseconds across papers or translating them into service throughput or monetary savings.

The audit reduces a further reproducibility risk by documenting the positional-index issue, but it does not retrospectively repair or certify historical runs. The old serialized model does not supply a fully verified manifest of its exact training rows. Its stored evaluation metrics and earlier benchmark summaries are therefore not merged with the new experiment. Any corrected production splitter or revised policy would require a new software version and renewed measurements. This article evaluates the supplied implementation and the separate research harness as recorded.

8 Conclusions

Promption provides a reusable setting for examining how heuristic rules, a lightweight text classifier, and enforcement policy interact at an LLM application's input boundary. The new experiments show that the learned component performs substantially better than the heuristic alone on the random holdouts, while OR enforcement adds a small amount of interception at a noticeably higher false-positive cost. The implemented three-state policy reduces some OR overblocking by routing inputs to output inspection, which transfers rather than resolves their downstream security status.

Source withholding changes the picture sharply. The implemented policy obtains 88.59 percent recall and 0.31 percent FPR for the Shomi28 holdout, but only 25.24 percent recall for NEPI and 17.37 percent for deepset. With jailbreak_llms withheld, recall reaches 96.76 percent alongside 80.09 percent FPR. These results establish that pooled random-holdout performance does not adequately describe transfer across the evaluated collections. They also show why both labels and enforcement actions must remain visible in a security evaluation.

The code audit identifies the configured lightweight backend, the difference between three-state routing and strict OR, and an error in the existing partition implementation. The fresh protocol refits features and classifiers only on training rows and supplies digests, predictions, metrics, and configuration hashes. Its strongest supported conclusion is an empirical account of input rejection and guarded routing under this corpus and software snapshot. Demonstrating reduced downstream compromise will require paired live-model and task-based experiments with explicit protected-property oracles.

9 Data and Software Availability

The accompanying reproducibility material contains the experimental harness, corpus digests, row-level predictions, metric tables, and an execution manifest. The evaluated repository commit is 8e13d8cffb80e79139758efea54687f3f353c275. The manifest records full SHA-256 values for the local processed corpus, label overrides, quarantine file, configuration, rules, and relevant source files. Public distribution pages for the upstream collections are given in [8]–[12]. These pages may change; recreating the exact local corpus requires the recorded project snapshot and curated CSVs. No persistent public archival identifier has been assigned to the accompanying software and result package at the time of preparation.

10 Research Scope and Responsible Use

The experiments analyze locally supplied prompt collections and execute classifier inference without sending their content to external model services. They do not involve new participant recruitment, live user traffic, or attacks against third-party systems. Corpus prompts are treated as research data rather than as instructions for the researcher or the experimental harness. Credentials and application configuration unrelated to the research are excluded from the accompanying material.

11 Use of Generative Artificial Intelligence

Generative artificial intelligence assisted with manuscript drafting, code inspection, and preparation of the experimental harness and document. Numerical results were produced by executing the recorded code against the supplied repository snapshot. The human authors are responsible for verifying the scientific interpretation, references, authorship, and final version. This disclosure does not assign authorship to an artificial-intelligence system.

References

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," *arXiv preprint* arXiv:2302.12173, 2023. https://doi.org/10.48550/arXiv.2302.12173

[2] J. Yi et al., "Benchmarking and defending against indirect prompt injection attacks on large language models," *arXiv preprint* arXiv:2312.14197, ver. 4, 2025. https://doi.org/10.48550/arXiv.2312.14197

[3] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," *arXiv preprint* arXiv:2403.02691, 2024. https://doi.org/10.48550/arXiv.2403.02691

[4] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," *arXiv preprint* arXiv:2406.13352, 2024. https://doi.org/10.48550/arXiv.2406.13352

[5] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," *arXiv preprint* arXiv:2404.13208, 2024. https://doi.org/10.48550/arXiv.2404.13208

[6] E. Debenedetti et al., "Defeating prompt injections by design," *arXiv preprint* arXiv:2503.18813, ver. 2, 2025. https://doi.org/10.48550/arXiv.2503.18813

[7] R. Bhagwatkar et al., "Indirect prompt injections: Are firewalls all you need, or stronger benchmarks?" *arXiv preprint* arXiv:2510.05244, ver. 2, 2026. https://doi.org/10.48550/arXiv.2510.05244

[8] verazuo, "jailbreak_llms," GitHub repository, accessed Oct. 2, 2026. [Online]. Available: https://github.com/verazuo/jailbreak_llms

[9] Vaibhav-GOAT, "nepi-prompts-dataset," Hugging Face dataset, accessed Oct. 2, 2026. [Online]. Available: https://huggingface.co/datasets/Vaibhav-GOAT/nepi-prompts-dataset

[10] Shomi28, "prompt-injection-dataset," Hugging Face dataset, accessed Oct. 2, 2026. [Online]. Available: https://huggingface.co/datasets/Shomi28/prompt-injection-dataset

[11] deepset, "prompt-injections," Hugging Face dataset, accessed Oct. 2, 2026. [Online]. Available: https://huggingface.co/datasets/deepset/prompt-injections

[12] yanismiraoui, "prompt_injections," Hugging Face dataset, accessed Oct. 2, 2026. [Online]. Available: https://huggingface.co/datasets/yanismiraoui/prompt_injections

[13] scikit-learn developers, "TfidfVectorizer," API documentation, accessed Oct. 2, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

[14] scikit-learn developers, "LogisticRegression," API documentation, accessed Oct. 2, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Appendix A Reproduction Protocol

The reproduction sequence begins with the recorded repository snapshot and its processed CSV. Before executing the harness, compare the corpus, override, quarantine, configuration, and heuristic-file hashes with manifest.json. A matching code commit alone is insufficient because data and serialized models can differ outside the tracked source tree. Preserve the original order of the processed rows; the normalized-duplicate policy deliberately retains the first row, and that choice can affect which source owns a shared prompt.

Run evidence/evaluate_article.py from the project environment. It locates the repository relative to its own path, calls the project's loader, selects the five source values, and constructs normalized identities. It then executes the three random holdouts and the five source holdouts in a fixed order. Classifier random state remains 42 in every fit; only the three partition seeds change. The training function returns in-memory vectorizers and classifiers, so the harness does not replace either model file used by the application.

For each experiment, metrics.csv supplies all four policy results, their confusion counts, the training-set size, and fit time. predictions.csv supplies one row per held-out example with its digest, source, label, heuristic score, positive probability, independent blocking flags, strict OR flag, and implemented decision. corpus_manifest.csv contains the retained identities and labels. decision_matrix.csv records selected boundary cases, including probabilities equal to 0.33 and 0.66 and unavailable ML signals. The execution log provides the fit sequence, and manifest.json supplies package versions and input fingerprints.

The partition check asserts that normalized digests in training and testing are disjoint. A reproducer should additionally confirm that each source-held-out test source is absent from the corresponding training frame, that each fit uses fresh feature transformers, and that the total counts match Table 3. Verify Table 5 by summing predictions for random_42. Verify Table 8 by grouping its decision column by label. Differences in numerical libraries can alter probabilities close to a boundary; such changes require reporting the reproduced environment rather than silently substituting new metrics.

This protocol fixes the observed dataset and policy rather than selecting their best results. To extend the study, create a new output directory and a new manifest. A changed threshold, rule file, preprocessing policy, language detector, or training collection defines another experiment. Mixing those outputs into the present tables would break the meaning of the paired comparisons. The same requirement applies to a production repair of the positional-index error: evaluate the repaired split separately and identify its new code version.

Appendix B Downstream Evaluation Contract

A subsequent end-to-end study should define its protected property before issuing model queries. For confidentiality, the oracle must examine whether protected information is delivered, including any chosen reconstruction or encoding cases. For tool integrity, the oracle must inspect the authorized operation and the resulting application state. For usefulness, a separate check must establish whether the legitimate task is completed. A refusal string alone is insufficient to measure all three properties, and blocking every input cannot be counted as useful task completion.

The paired design should run unfiltered and filtered conditions over the same attack scenarios with controlled model configuration. Record model identity, generation settings, number of attempts, context, action path, guard verdict, output, and the exact denominator of each metric. Retry policies must be fixed and failed requests must remain visible. If stochastic generation is repeated, report both the number of unique scenarios and the number of query attempts so that repeated samples are not mistaken for independent attacks.

The guarded path needs a distinct ablation. Compare input-only screening, input screening with output inspection, and input screening with output inspection and tool authorization. Include legitimate tasks under each condition. This design can estimate whether the 89 guarded malicious inputs in the present seed-42 partition are stopped downstream and whether the 589 guarded benign inputs still complete useful work. It should also evaluate cold startup, concurrent request load, and streaming delivery separately from the warm scalar timing reported here.

Adaptive and indirect attacks require more context than a stand-alone text label. Freeze the defense before developing attacks against it, preserve the trust labels of user requests and retrieved content, and record what tools the agent can access. A protected-property oracle should remain outside the attacker-controlled text. This extension would connect the current input-screening evidence to the task and environment perspective of agent benchmarks without treating the two observation units as equivalent.